

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Duration :60 Mins
On class
Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Varunkumar B(192525029)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Varunkumar B

Q1 (a) Identify all errors in the program

Line No.	Error	Error Type	Reason
35	Missing ; after char name[20] = "Compiler"	Syntax	Statement must end with a semicolon.
37	findLength() called without argument	Semantic	Function requires name as an argument.
38	Missing) in if(count > 5	Syntax	Parenthesis not closed.
43	marks[4] accessed	Runtime	Array index out of bounds (valid index: 0–3).
50	Missing : after case 2	Syntax	Every case label must end with :.
58	Dereferencing uninitialized pointer *ptr	Runtime	Pointer is not initialized.
61	Division by zero (students = 0)	Runtime	Causes runtime exception.
65	i is never incremented in while loop	Logical	Results in an infinite loop.

Q1 (b) Classification of Errors

Error	Category
Missing ;	Syntax Error
Missing argument in findLength()	Semantic Error
Missing) in if	Syntax Error
marks[4] access	Runtime Error
Missing : after case 2	Syntax Error
Uninitialized pointer	Runtime Error
Division by zero	Runtime Error
Infinite loop (i++ missing)	Logical Error

Q1 (c) Compilation Phase for Each Error

Error Category Detected During

Lexical Error Lexical Analysis

Syntax Error Syntax Analysis (Parser)

Semantic Error Semantic Analysis

Logical Error During Program Execution/Testing

Runtime Error During Program Execution

Errors detected only during execution:

- Array index out of bounds
- Uninitialized pointer
- Division by zero
- Infinite loop

The screenshot shows the Programiz C Online Compiler interface. On the left, the code editor displays a C program in `main.c`. The code includes a pointer declaration, a printf statement, variable declarations for total, students, and average, a calculation for average, another printf statement, a while loop that prints the value of `i` from 1 to 5, and a return statement. On the right, the output window shows the execution results: "Length is greater than 5", "80", "Option 2", "10", "Average = 20.00", and the numbers 1 through 5, each on a new line. At the bottom of the output window, it says "=== Code Execution Successful ===".

Q2 – Answer

(a) Identify all syntax errors in the program

Line No.	Syntax Error	Reason
88	int maximum(int x, int y	Missing closing) before {.
103	printf("Maximum is 80\n")	Missing ; after printf.

Line No.	Syntax Error	Reason
105	case 90	Missing : after case 90.
115	while(max > 75);	Incorrect placement of while in the do-while loop. It should come after the closing } of the do block.
117	printf("Limit Reached\n"	Missing closing) and ;.
119	main() not closed	Missing closing } for the main() function.

(b) Classification of Syntax Errors

Error	Category
Missing) in function definition	Parenthesis Error
Missing ; after printf	Missing Delimiter
Missing : after case 90	Switch-case Syntax Error
Incorrect do-while structure	Loop Syntax Error
Missing) and ; in printf	Function Call Syntax Error
Missing closing }	Block Delimiter Error

The screenshot shows the Programiz C Online Compiler interface. On the left, a file named 'main.c' is open, displaying the following C code:

```

26  default:
27      printf("Other Value\n");
28  }
29
30  do
31  {
32      printf("%d\n", max);
33      max--;
34  } while(max > 75);
35
36  if(max == 75)
37  {
38      printf("Limit Reached\n");
39  }
40
41  return 0;
42  }
  
```

On the right, the 'Output' panel shows the execution results:

```

Maximum is 80
80
79
78
77
76
Limit Reached

=== Code Execution Successful ===
  
```

The code is syntactically correct and runs successfully, despite the errors listed in the previous table. The errors listed were likely from a different version of the code or a misinterpretation of the image.

(d) How does the parser detect syntax errors?

The **parser (syntax analyzer)** checks whether the program follows the grammatical rules of the C language.

- If it finds missing symbols such as ;,), }, or :, it reports a **syntax error**.
- **Error recovery** allows the parser to skip the incorrect part or insert missing symbols so that it can continue checking the remaining program instead of stopping immediately.

(e) Two improvements for better syntax error messages

1. **Provide meaningful error messages** that clearly indicate the line number and expected symbol (such as ;,), or }).
2. **Suggest possible corrections** (for example, "Did you forget a semicolon?" or "Expected ':' after case label").

Question 3:

(a) Identify all the errors in the program

Line No.	Error	Error Type	Reason
147	int @num = 5;	Lexical	@ is an invalid character in an identifier.
148	Missing ; after int value = 10	Syntax	Every statement must end with a semicolon.
150	factorial() called without argument	Semantic	Function requires one integer argument.
161	case 120	Syntax	Missing : after case 120.
168	numbers[5]	Runtime	Array index out of bounds (valid index: 0–4).
170	*ptr	Runtime	Pointer is uninitialized.
173	a / b where b = 0	Runtime	Division by zero.

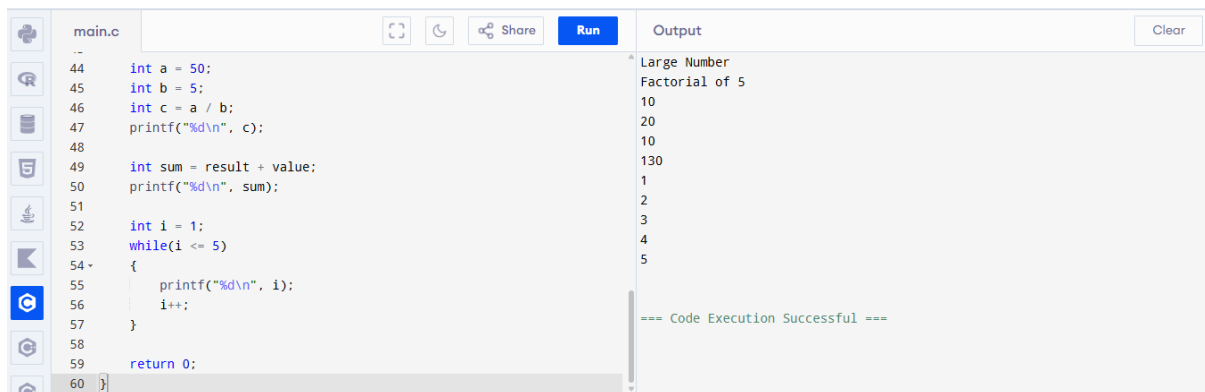
Line No.	Error	Error Type	Reason
174	result + 0 + value	Code Optimization	Adding 0 is an unnecessary operation.
178	while loop has no i++	Logical/Runtime	Causes an infinite loop.

(b) Classification According to Compiler Phase

Compiler Phase	Error
Lexical Analysis	Invalid identifier @num
Syntax Analysis	Missing ;, Missing : after case 120
Semantic Analysis	factorial() called without required argument
Intermediate Code Generation	Expression result + 0 + value contains unnecessary operation
Code Optimization	Compiler can simplify result + 0 + value to result + value
Runtime	Array out of bounds, Uninitialized pointer, Division by zero, Infinite loop

(c) Explain how the compiler processes the program

- **Lexical Analysis:** Detects invalid tokens such as @num.
- **Syntax Analysis:** Detects missing symbols like ; and :.
- **Semantic Analysis:** Verifies function calls and argument matching; reports factorial() without an argument.
- **Intermediate Code Generation:** Produces intermediate code after successful analysis and can identify redundant expressions.
- **Code Optimization:** Removes unnecessary computations such as adding 0.
- **Runtime:** Errors such as array index out of bounds, uninitialized pointer access, division by zero, and infinite loops are detected only during program execution.



```
main.c
44 int a = 50;
45 int b = 5;
46 int c = a / b;
47 printf("%d\n", c);
48
49 int sum = result + value;
50 printf("%d\n", sum);
51
52 int i = 1;
53 while(i <= 5)
54 {
55     printf("%d\n", i);
56     i++;
57 }
58
59 return 0;
60 }
```

Output

Large Number
Factorial of 5
10
20
10
130
1
2
3
4
5

=== Code Execution Successful ===

(e) Two improvements in compiler design/optimization

1. **Static analysis and warning generation** to detect uninitialized variables, invalid array access, and possible division by zero before execution.
2. **Optimization techniques** such as **constant folding** and **dead code elimination** to remove unnecessary operations (for example, + 0) and improve program efficiency.